

MANUEL D'UTILISATION

Gunther.ino

« Mode simplifié »

2026-2027

TABLE DES MATIÈRES

L'ASSOCIATION ET LES GRANDS ACTEURS DU PROJET 3
PRÉSENTATION 4
VUE D'ENSEMBLE 5
ALIMENTATION 6
PREMIER PROGRAMME 7
BRANCHEMENTS 8
DÉMARRAGE ET AUTO-TEST 9
LED, SWITCHS, JACK, BUZZER, PILE 10
CAPTEURS 11
SUIVI DU VOL 12
SERVOS 13
ENREGISTREMENT SUR LA CARTE SD 14
TÉLÉMÉTRIE 15
PROGRAMME DE VOL TYPE 16
EXEMPLES FOURNIS 17
EN CAS DE PROBLÈME 18
CONTACT ET VERSION 19



L'ASSOCIATION ET LES GRANDS ACTEURS DU PROJET

Créée en 1986, l'Association Aérospatiale Air ESIEA est engagée depuis près de 40 ans dans les événements organisés par l'ANSTJ puis par Planète Sciences. Ayant pour objet la promotion et l'organisation d'activités techniques et pédagogiques autour des domaines des sciences et de l'aérospatial, elle est amenée à participer au C'Space organisé par Planète Science ainsi qu'à des événements comme EuRoC et d'autres campagnes indépendantes.

L'équipe du projet sera donc épaulée tout au long de l'année par Air ESIEA qui elle-même bénéficie du soutien opérationnel de ses partenaires. Le plus important d'entre eux restant l'école d'ingénieur à laquelle elle est rattachée : l'ESIEA.

L'ESIEA fournit à Air ESIEA depuis sa création des locaux à titre gracieux ainsi qu'une dotation pécuniaire qu'elle laisse à sa charge de répartir entre ses nombreux projets. La philosophie globale reste de faire des fusées aussi peu chères que possible, sans jamais compromettre la sécurité du public ou de la mission.

PRÉSENTATION

Gunther.ino est la carte électronique des minifusées d'Air ESIEA. Elle détecte le décollage et l'apogée, commande l'éjection du parachute (trappe ou coiffe) et enregistre les données du vol sur une carte micro SD.

En mode simplifié, tout passe par la bibliothèque **Gunther** fournie avec la carte : une seule ligne `#include <Gunther.h>` et chaque action devient une fonction (`Gunther.altitude()`, `Gunther.apogee()`, `Gunther.servoOpen(1)`...). Les capteurs, la détection des phases du vol et l'enregistrement sont déjà écrits et testés.

Caractéristiques

Élément	Détail
Microcontrôleur	ATSAMD51J20A : Cortex-M4F à 120 MHz, 1 Mo de flash, 256 Ko de RAM
Baromètre	BMP390 (I2C, adresse 0x76) : pression, température, altitude. BMP388 et SPL07-003 acceptés
Centrale inertielle	BMI088 (SPI) : accéléromètre ±24 g et gyroscope ±2000 °/s
Stockage	Lecteur de carte micro SD (SPI), avec détection de carte
Sorties	4 LED, 4 servos, 1 buzzer piézo
Entrées	4 switchs, 1 jack de décollage, mesure de la tension pile
Extensions	1 bus I2C, 1 bus SPI avec un CS libre, 1 UART
Alimentation	Pile 9 V (7 à 12 V) ou USB-C
Programmation	USB-C avec l'Arduino IDE (bootloader UF2), ou sonde SWD

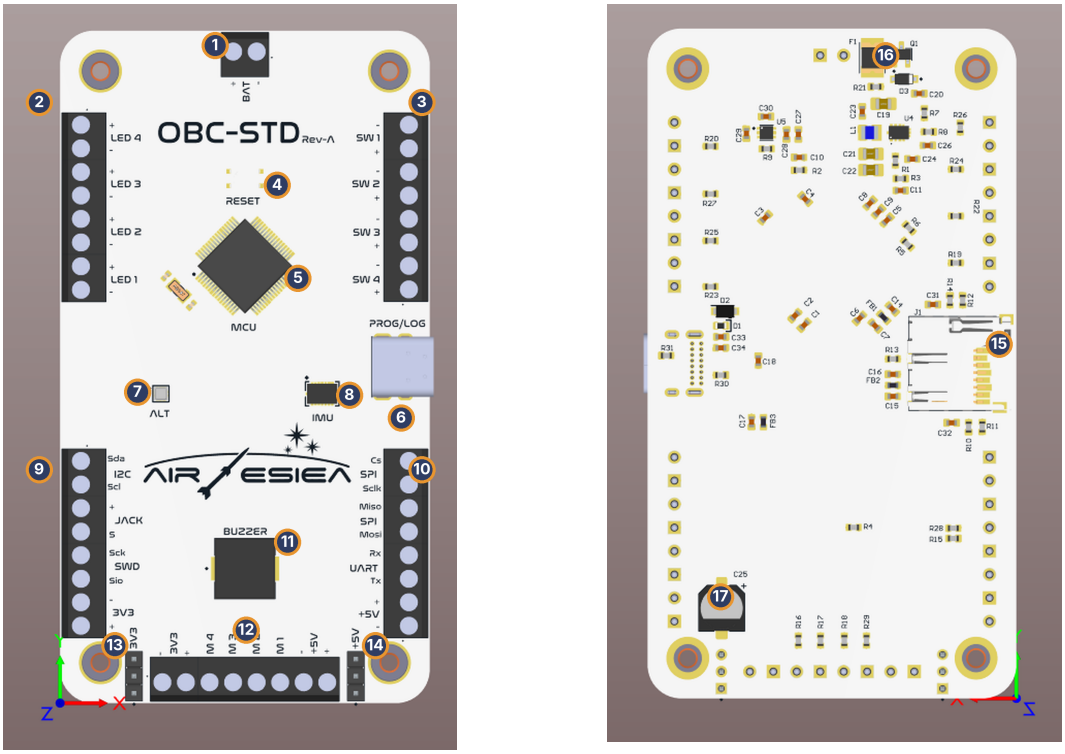
Deux façons de programmer la carte

	Mode simplifié	Mode classique
Principe	Bibliothèque `Gunther` : une fonction par action (`Gunther.altitude()`, `Gunther.apogee()`...)	Code Arduino standard, comme sur une Arduino Nano, avec les noms des broches de la carte
Pour qui	Premiers vols, équipes qui veulent un programme fiable rapidement	Ceux qui veulent écrire eux-mêmes la lecture des capteurs et la logique du vol
Exemples	**Fichier › Exemples › Gunther**	**Fichier › Exemples › Gunther Classique**
Manuel	Manuel d'utilisation, mode simplifié	Manuel d'utilisation, mode classique

Ce manuel couvre le mode **simplifié**. Les deux modes peuvent être mélangés dans le même programme : voir le manuel du mode classique pour l'autre partie.

VUE D'ENSEMBLE

Les numéros sur les images renvoient au tableau en dessous.



Dessus et dessous de la carte

N°	Élément	Rôle
1	BAT	Entrée pile (+ et −)
2	LED 1 à LED 4	Sorties LED
3	SW 1 à SW 4	Entrées switchs
4	RESET	Redémarrage (double appui = bootloader)
5	MCU	Microcontrôleur ATSAMD51J20A
6	PROG/LOG	USB-C : programmation et moniteur série
7	ALT	Baromètre BMP390
8	IMU	Centrale inertielle BMI088
9	Bornier gauche	I2C, JACK, SWD, 3V3
10	Bornier droit	SPI, UART, +5V
11	BUZZER	Buzzer piézo
12	Bornier du bas	3V3, servos M1 à M4, +5V
13	Barrette 3V3	Alimentation 3,3 V
14	Barrette +5V	Alimentation 5 V des servos
15	Carte SD	Lecteur micro SD (dessous)
16	Protection	Fusible réarmable 1,5 A et anti-inversion de la pile
17	Condensateur	Réserve d'énergie du +5V (220 µF)

ALIMENTATION

La pile arrive sur **BAT**, passe par la protection, puis un convertisseur fait le **5 V** et un régulateur fait le **3,3 V** à partir du 5 V. L'USB-C fournit aussi le 5 V, à travers une diode : les deux peuvent être branchés en même temps.

Rail	Où le trouver	Limite
BAT	Bornier BAT	7 à 12 V (pile 9 V). Fusible 1,5 A, protégé contre l'inversion
+5V	Bornier droit, bornier du bas, barrette +5V	2 A au total, partagé avec le 3,3 V
3V3	Bornier gauche, bornier du bas, barrette 3V3	500 mA au total, dont environ 50 mA pour la carte
USB-C	Prise PROG/LOG	500 mA (limite du port USB du PC)

La tension de la pile se lit directement en volts avec `Gunther.batteryVoltage()`.

> Les borniers +5V et 3V3 sont reliés directement aux alimentations de la carte : un court-circuit entre + et − coupe toute la carte.

PREMIER PROGRAMME

Installation du paquet Arduino

1. **Fichier › Préférences**, puis coller ce lien dans **URL de gestionnaire de cartes supplémentaires** :

```
https://raw.githubusercontent.com/Arkyos03/Gunther.ino-Arduino/main/package_gunther_index.json
```

2. **Outils › Type de carte › Gestionnaire de cartes** : chercher **Gunther.ino** et installer.
3. Choisir la carte **Gunther.ino (Rev A)** et le port USB.
4. Écrire le programme puis cliquer sur **Téléverser**, comme pour une carte Arduino.

Si le téléversement ne marche plus (programme planté), un **double appui sur RESET** met la carte en mode bootloader : la LED 1 pulse et un lecteur **GUNTHERBOOT** apparaît. Il suffit de relancer le téléversement.

Structure d'un programme

```
#include <Gunther.h>


void setup() {
  Serial.begin(115200);        // moniteur série (optionnel)
  Gunther.begin();             // démarre la carte et les capteurs
  Gunther.selfTest();          // 3 bips = tout est OK
}

void loop() {
  Gunther.update();            // lit les capteurs, suit le vol
  // ... ton programme ...
}
```

- `Gunther.begin()` se met toujours au début de `setup()`. Il configure les broches, démarre le baromètre et la centrale inertielle et mesure la pression au sol (l'altitude 0).
- `Gunther.update()` se met au début de `loop()`. Les autres fonctions l'appellent aussi : l'oublier ne bloque rien, mais l'appeler souvent rend la détection du vol plus précise.
- Éviter les longs `delay()` dans `loop()` pendant un vol : 10 à 20 ms au maximum.

BRANCHEMENTS

En mode simplifié, les éléments du bornier sont désignés par leur numéro dans les fonctions (`Gunther.led(2, true)` pour la LED 2, `Gunther.servoOpen(1)` pour le servo M1…).

Repère sur la carte	Dans le code	Branchement
LED 1 à LED 4	`Gunther.led(1..4, ...)`	LED directement entre + et − (résistance 330 Ω déjà sur la carte)
SW 1 à SW 4	`Gunther.switchClosed(1..4)`	Interrupteur entre + et − : fermé = `true`
JACK	`Gunther.jackPlugged()`, `Gunther.liftoff()`	Jack entre + (3,3 V) et S : en place = branché, arraché = décollage
M1 à M4	`Gunther.servoOpen(1..4)`…	Servo : signal sur la broche, alimentation sur +5V et GND
BUZZER	`Gunther.beep()`	Déjà sur la carte
BAT	`Gunther.batteryVoltage()`	Pile 9 V
Carte SD	`Gunther.logBegin()`…	Carte micro SD formatée en FAT32 ou exFAT
Tx / Rx	`Serial1`, `Gunther.printFlight(Serial1)`	Module radio ou GPS (niveaux 3,3 V)
Sda / Scl, SPI	–	Capteurs externes : voir le manuel du mode classique

DÉMARRAGE ET AUTO-TEST

Fonction	Ce qu'elle fait
`Gunther.begin()`	Démarre la carte, les capteurs et mesure l'altitude du sol. Renvoie `false` si un capteur ne répond pas (la carte fonctionne quand même)
`Gunther.update()`	Lit les capteurs et suit le vol (au plus une fois par milliseconde)
`Gunther.selfTest()`	Vérifie baromètre, centrale, carte SD et pile, affiche un rapport sur le moniteur série et bipe le résultat
`Gunther.selfTest(false, false)`	Même test sans exiger la carte SD ni la pile (essais sur USB)
`Gunther.enterBootloader()`	Redémarre la carte en mode bootloader

Code des bips

Bips	Résultat	Valeur renvoyée
3 bips courts	Tout est OK	`GUNTHER_OK`
1 bip long + 1 court	Baromètre absent	`GUNTHER_ERR_BARO`
1 bip long + 2 courts	Centrale inertielle absente	`GUNTHER_ERR_IMU`
1 bip long + 3 courts	Carte SD absente ou illisible	`GUNTHER_ERR_SD`
1 bip long + 4 courts	Pile sous 7 V	`GUNTHER_ERR_BATTERY`

```
if (Gunther.selfTest() != GUNTHER_OK) {
  while (true) { Gunther.ledToggle(4); delay(200); }   // on ne part pas en vol
}
```

LED, SWITCHS, JACK, BUZZER, PILE

Fonction	Ce qu'elle fait
`Gunther.led(n, true/false)`	Allume ou éteint la LED n (1 à 4)
`Gunther.ledToggle(n)`	Inverse l'état de la LED n
`Gunther.ledBrightness(n, 0-255)`	Luminosité de la LED 1 ou 2
`Gunther.allLeds(true/false)`	Toutes les LED d'un coup
`Gunther.switchClosed(n)`	`true` si le switch n (1 à 4) est fermé
`Gunther.jackPlugged()`	`true` si le jack est en place
`Gunther.beep(fréquence, durée)`	Bip sans bloquer le programme (4000 Hz, 100 ms par défaut)
`Gunther.beepBlocking(fréquence, durée)`	Bip qui attend la fin du son
`Gunther.beepCount(n)`	n bips courts
`Gunther.beepAltitude()`	Bipe l'altitude max chiffre par chiffre (voir Suivi du vol)
`Gunther.batteryVoltage()`	Tension de la pile en volts
`Gunther.lowBattery()`	`true` sous 7 V (`Gunther.lowBattery(7.5)` pour un autre seuil)

CAPTEURS

Baromètre

Le baromètre est reconnu automatiquement au démarrage : **BMP390**, **BMP388/BMP384** (Bosch) ou **SPL07-003** (vendu sous le nom « BMP388 » par certains fabricants, même empreinte). Il est lu à 50 Hz.

Fonction	Ce qu'elle fait
`Gunther.altitude()`	Altitude en mètres par rapport au sol (0 au démarrage)
`Gunther.altitudeSea()`	Altitude au-dessus du niveau de la mer (1013,25 hPa par défaut)
`Gunther.pressure()`	Pression en pascals
`Gunther.temperature()`	Température du capteur en °C
`Gunther.verticalSpeed()`	Vitesse verticale en m/s (positive en montée, filtrée)
`Gunther.setGroundLevel()`	Refait le zéro d'altitude ici (déjà fait par `begin()`)
`Gunther.baroOK()` · `Gunther.baroName()`	Baromètre détecté, et son nom

Centrale inertielle BMI088

Accéléromètre ±24 g et gyroscope ±2000 °/s, lus à chaque `Gunther.update()`.

Fonction	Ce qu'elle fait
`Gunther.accel(x, y, z)`	Les 3 accélérations en m/s² dans x, y, z
`Gunther.gyro(x, y, z)`	Les 3 vitesses de rotation en °/s
`Gunther.accelX()` … `Gunther.gyroZ()`	Un seul axe
`Gunther.gForce()`	Accélération totale en g (1 au repos)
`Gunther.imuOK()`	Centrale détectée

```
float ax, ay, az;
Gunther.accel(ax, ay, az);
Serial.printf("%.1f m  %.2f %.2f %.2f\n", Gunther.altitude(), ax, ay, az);
```

SUIVI DU VOL

La bibliothèque suit les phases du vol toute seule. `Gunther.state()` donne la phase en cours :

Phase	Valeur	Passage à la phase suivante
Au sol	`GUNTHER_GROUND`	Jack arraché pendant 20 ms (ou accélération, si réglée)
Montée	`GUNTHER_ASCENT`	Altitude 3 m sous le maximum pendant 5 mesures, au moins 1 s après le décollage (ou sécurité de temps)
Descente	`GUNTHER_DESCENT`	Vitesse verticale sous 1 m/s pendant 3 s
Posée	`GUNTHER_LANDED`	–

Fonction	Ce qu'elle fait
`Gunther.liftoff()`	`true` depuis le décollage
`Gunther.apogee()`	`true` depuis l'apogée
`Gunther.landed()`	`true` une fois la fusée posée
`Gunther.flightTime()`	Temps depuis le décollage en ms (0 au sol)
`Gunther.maxAltitude()`	Altitude maximale atteinte en m
`Gunther.resetFlight()`	Retour à la phase « au sol » (essais)

Réglages

À appeler dans `setup()`, après `Gunther.begin()`.

Fonction	Défaut	Effet
`Gunther.setApogeeDrop(m)`	3 m	Baisse d'altitude qui confirme l'apogée
`Gunther.setApogeeMinTime(ms)`	1000 ms	Pas d'apogée avant ce temps de vol
`Gunther.setApogeeTimeout(ms)`	0 (aucune)	Sécurité : apogée forcée après ce temps de vol
`Gunther.setLiftoffAccel(g)`	0 (aucune)	Décollage aussi détecté au-delà de cette accélération pendant 50 ms
`Gunther.setLandingTime(ms)`	3000 ms	Durée d'immobilité qui confirme l'atterrissage

> La sécurité de temps (`setApogeeTimeout`) est fortement conseillée : elle ouvre le parachute même si le baromètre est perturbé. Choisir une durée un peu après l'apogée prévue par la simulation.

Retrouver l'altitude après le vol

`Gunther.beepAltitude()` bipe l'altitude maximale chiffre par chiffre : 437 m donnent 4 bips, une pause, 3 bips, une pause, 7 bips. Un bip long vaut 0. On peut aussi donner une valeur : `Gunther.beepAltitude(125)`.

SERVOS

Fonction	Ce qu'elle fait
`Gunther.servoAngles(n, fermé, ouvert)`	Angles du servo n en degrés (0 et 90 par défaut)
`Gunther.servoClose(n)`	Met le servo n en position fermée
`Gunther.servoOpen(n)`	Met le servo n en position ouverte
`Gunther.servoWrite(n, angle)`	Angle quelconque (0 à 180°)
`Gunther.servoDetach(n)`	Coupe le signal : le servo ne force plus

```
Gunther.servoAngles(1, 10, 100);   // trappe fermée à 10°, ouverte à 100°
Gunther.servoClose(1);
...
if (Gunther.apogee()) Gunther.servoOpen(1);
```

ENREGISTREMENT SUR LA CARTE SD

`Gunther.logBegin()` crée un nouveau fichier à chaque démarrage : **vol_001.csv**, **vol_002.csv**... Un vol n'est jamais écrasé. Les données sont écrites sur la carte au moins toutes les 0,5 s : en cas de coupure, on perd au plus la dernière demi-seconde.

Fonction	Ce qu'elle fait
`Gunther.logBegin()`	Nouveau fichier avec l'en-tête complet. Renvoie `false` sans carte SD
`Gunther.logBegin("essai", "temps;alt")`	Autre préfixe (7 lettres max) et en-tête personnalisé
`Gunther.logFlight()`	Une ligne complète (voir colonnes ci-dessous)
`Gunther.log(a, b, c...)`	Une ligne avec le temps puis les valeurs données
`Gunther.logText("texte")`	Une ligne de texte (événement)
`Gunther.logFlush()` · `Gunther.logClose()`	Forcer l'écriture, fermer le fichier
`Gunther.setLogSeparator(',')`	Séparateur des colonnes (`;` par défaut, pour Excel en français)
`Gunther.sdCardPresent()` · `Gunther.logFileName()`	Carte insérée et lisible (testée au plus une fois par seconde), nom du fichier en cours

Colonnes de Gunther.logFlight()

Colonne	Unité
temps_ms	ms depuis le démarrage
etat	0 au sol, 1 montée, 2 descente, 3 posée
altitude_m · pression_Pa · temperature_C	m (par rapport au sol), Pa, °C
ax_ms2 · ay_ms2 · az_ms2	m/s²
gx_dps · gy_dps · gz_dps	°/s
batterie_V	V

TÉLÉMÉTRIE

`Gunther.printFlight(port)` envoie la même ligne que `Gunther.logFlight()` sur un port série : `Serial` pour le moniteur ou le traceur série, `Serial1` pour un module radio branché sur Tx / Rx. `Gunther.printFlightHeader(port)` envoie la ligne d'en-tête.

```
Serial1.begin(9600);                 // vitesse du module radio
...
static uint32_t dernier = 0;
if (millis() - dernier >= 200) {     // 5 lignes par seconde
  dernier = millis();
  Gunther.printFlight(Serial1);
}
```

Une radio lente (9600 bauds) envoie environ 10 lignes par seconde au maximum : ne pas appeler `printFlight` à chaque tour de `loop()`.

PROGRAMME DE VOL TYPE

Exemple **10_Atterrissage_Bips** : auto-test, enregistrement, ouverture à l'apogée avec sécurité, puis l'altitude bipée en boucle une fois au sol.

```
#include <Gunther.h>

void setup() {
  Gunther.begin();
  Gunther.servoAngles(1, 0, 90);
  Gunther.servoClose(1);
  Gunther.setApogeeTimeout(8000);  // ouverture forcée à 8 s
  if (Gunther.selfTest() != GUNTHER_OK) {
    while (true) { Gunther.ledToggle(4); delay(200); }
  }
  Gunther.logBegin();
  Gunther.led(1, true);            // prête
}

void loop() {
  Gunther.update();
  Gunther.logFlight();
  if (Gunther.apogee()) Gunther.servoOpen(1);
  if (Gunther.landed()) {
    Gunther.logClose();
    Gunther.servoDetach(1);
    while (true) { Gunther.beepAltitude(); delay(5000); }
  }
  delay(10);
}
```

EXEMPLES FOURNIS

Dans **Fichier › Exemples › Gunther** :

Exemple	Contenu
01_Clignotant	Faire clignoter les LED
02_Buzzer	Un bip toutes les secondes
03_Ejection_Servo	Ouverture de la trappe à l'apogée, sécurité à 8 s
04_Batterie	Tension de la pile sur le moniteur série
05_Scanner_I2C	Liste les composants présents sur le bus I2C
06_Test_Carte	Capteurs, carte SD, pile, puis LED, switchs et jack en continu
07_Vol_Complet	Vol complet avec enregistrement et ouverture à l'apogée
08_Capteurs	Affiche baromètre et centrale sur le moniteur série
09_Test_Fabrication	Test guidé d'une carte neuve, élément par élément
10_Atterrissage_Bips	Vol complet, atterrissage et altitude bipée
11_Telemetrie	Données de vol envoyées par radio sur Serial1

EN CAS DE PROBLÈME

Problème	Solution
Le téléversement échoue	Double appui sur RESET (la LED 1 pulse), puis relancer le téléversement
Pas de port USB dans l'IDE	Câble USB de charge seule : en prendre un qui transmet les données. Sinon double appui sur RESET
1 bip long + 1 bip court	Baromètre non détecté : vérifier la soudure du capteur ALT
1 bip long + 3 bips courts	Carte SD absente, mal insérée ou non formatée (FAT32 ou exFAT)
Le décollage se déclenche tout seul	Jack mal enfoncé ou absent : `Gunther.jackPlugged()` doit valoir `true` au sol
L'apogée est détectée trop tôt	Augmenter `setApogeeDrop` ou `setApogeeMinTime` ; protéger le baromètre du vent et de la lumière
Le servo tremble ou chauffe	Pile trop faible pour le +5V, ou angle hors des butées : réduire l'angle
Rien sur le moniteur série	`Serial.begin(115200)` dans `setup()`, et choisir le bon port

CONTACT ET VERSION

Références

- Code et paquet Arduino : https://github.com/Arkyos03/Gunther.ino-Arduino
- Pour plus d'information sur Air ESIEA : https://airesiea.org

Contact

Pour toute question sur la carte, contactez Arkyos03 ou l'association :

Courriel : air-esiea@et.esiea.fr

Date et version

Manuel mode simplifié : version 1.2, 26/10/2026. Carte : Gunther.ino Rev A. Paquet Arduino : 0.1.5.